

PROJET AUTOMATES FINIS ET EXPRESSIONS RATIONNELLES

PRÉSENTÉ À

Olga MELEKHOVA

PRÉSENTÉ PAR

Tom GUERRIER

Mael LECARPENTIER

Theo UNG

Lucas NGUYEN

Julien BLANCHARD

SOMMAIRE

PROJET AUTOMATE

01 - Introduction

3

02 - Planification des tâches

4

03 - Explication du code :

- a) Stockage de l'automate
- b) Affichage de l'automate
- c) Lecture du fichier texte de l'automate
- d) Affichage de la table de transition
- e) Vérifications sur l'automate
- f) Standardisation
- g) Détermination de l'automate
- h) Compléter l'automate
- i) Automate complémentaire

5 - 11

04 - Test du code

12

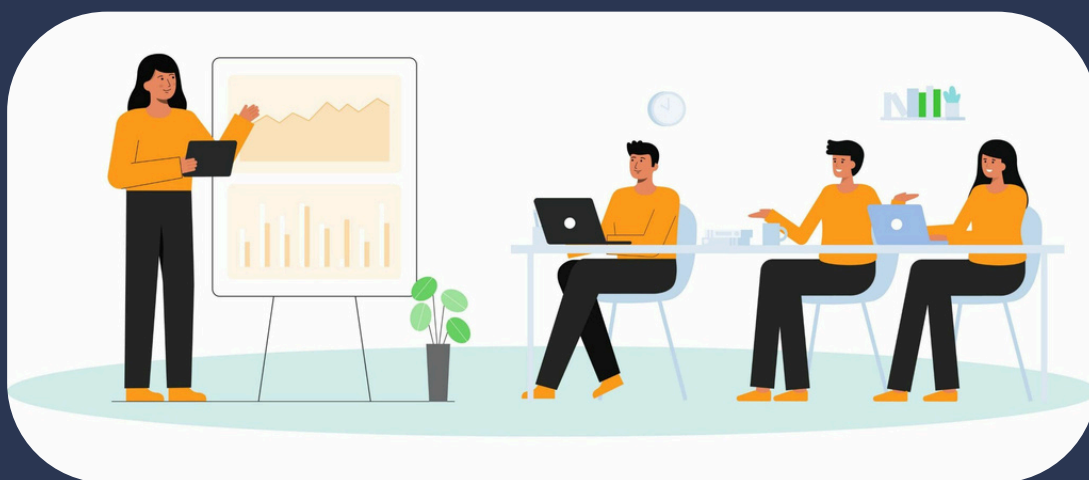
01 - Introduction du projet

Réaliser un programme qui permet de:

- lire des automates finis.
- afficher les détails de notre automate
- afficher la table de transition de notre automate
- effectuer des vérifications sur sa nature
- le modifier en cas de besoin (standardiser, compléter, déterminer)
- afficher le complémentaire de notre automate

02 - Planification des tâches

DATE	TÂCHES RÉALISÉES
05/04/2024	• Analyse du sujet
08/04/2024	• Création de la base du code • Choix du langage de programmation (en C) • Répartition des tâches
15/04/2024 au 21/04/2024	• Changement de langage de programmation (en Python) • Reprise du projet depuis le début
21/04/2024	• Dépôt du projet sur Moodle • Rédaction du rapport
22/04/2024	• Finalisation du rapport • Finalisation du code



03 - Explication du code

a - Stockage de l'automate

```
def __init__(self, fichier):
    self.fichier = fichier
    self.transitions = []
    self.etats = []
    self.nb_symb = 0
    self.nb_etats = 0
    self.init_etats = []
    self.nb_init_etats = 0
    self.term_etats = []
    self.nb_term_etats = 0
    self.nb_transitions = 0
    self.symb = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5, 'f': 6, 'g': 7, 'h': 8, 'i': 9, 'j': 10, 'k': 11, 'l': 12,
                 'm': 13, 'n': 14, 'o': 15, 'p': 16, 'q': 17, 'r': 18, 's': 19, 't': 20, 'u': 21, 'v': 22, 'w': 23,
                 'x': 24, 'y': 25, 'z': 26}
```

b - Affichage de l'automate

```
def afficher_automate(self):
    print("Nombre de caractere(s) dans l'alphabet :", self.nb_symb)
    print("Nombre d'etat(s) :", self.nb_etats)
    print("Nombre d'etat(s) initial :", self.nb_init_etats)
    print("Etat(s) initial :", self.init_etats)
    print("Nombre d'etat(s) terminal :", self.nb_term_etats)
    print("Etat(s) terminal :", self.term_etats)
    print("Nombre de transition(s) :", self.nb_transitions)
    print("Transition(s) :", self.transitions)
```

c - Lecture du fichier texte de l'automate

```
def lecture_automate(self):
    file = open(f"./{self.fichier}", "r")
    self.nb_symb = int(file.readline())
    self.nb_etats = int(file.readline())
    self.etats = []

    init_etats = file.readline()
    self.nb_init_etats = int(init_etats[0])
    self.init_etats = init_etats[1:].split()

    self.term_etats = file.readline()
    self.nb_term_etats = int(self.term_etats[0])
    self.term_etats = self.term_etats[1:].split()

    self.nb_transitions = int(file.readline())

    for _ in range(self.nb_transitions):
        etat1, etat2 = "", ""
        line = file.readline()
        j = 0

        while (line[j] not in self.symb):
            etat1 += line[j]
            j += 1
        k = j
        j += 1

        while (j < len(line)):
            if (line[j] != "\n"):
                etat2 += line[j]
                j += 1

        if (etat1 not in self.etats):
            self.etats.append(etat1)

        if (etat2 not in self.etats):
            self.etats.append(etat2)

        self.transitions.append((etat1, line[k], etat2))
```

```
2
5
2 1 3
2 2 4
4
1a2
1b0
0a0
0b0
```

d - Affichage de la table de transition

```
def afficher_table_transition(self):
    transition_table = self.transition_vers_tableau()
    symboles = self.get_symboles()

    print("          + " + "-----+" * self.nb_symb)
    print("          |", end="")
    for symbole in symboles:
        print(" {:^9} |".format(symbole), end="")
    print()

    print("----+-----+" + "-----+" * self.nb_symb)

    for i in range(self.nb_etats):

        if str(transition_table[i][0]) in self.term_etats and str(transition_table[i][0]) in self.init_etats:
            print("ES|", end="")
        elif str(transition_table[i][0]) in self.term_etats:
            print(" S|", end="")
        elif str(transition_table[i][0]) in self.init_etats:
            print(" E|", end="")
        else:
            print("  |", end="")

        print(' {:>8} |'.format(transition_table[i][0]), end="")
        print_strings = []

        for row in transition_table[i][1:]:
            if len(row) > 1:
                print_strings.append('_{:^9}_{:~9}'.format(*row))
            else:
                print_strings.append('_{:^9}_{:~9}'.format(*row))

        for value in print_strings:
            print(value, end="")

        print()
        print("----+-----+" + "-----+" * self.nb_symb)
```

e - Vérifications sur l'automate

```
def est_standard(self):
    if self.nb_init_etats > 1:
        return False
    return not (self.transition_vers_etat_init())

def transition_vers_etat_init(self):
    return any(i[2] in self.init_etats for i in self.transitions)

def est_deterministe(self):
    transition_table = self.transition_vers_tableau()
    if self.nb_init_etats > 1:
        return False

    return not any(len(transition[i]) > 1 for transition in transition_table for i in range(1, len(transition)))

def est_complet(self):
    transition_table = self.transition_vers_tableau()
    return all([''] not in row[1:] for row in transition_table) and all(
        [''] not in row[1:] for row in transition_table)
```

f - Standardisation

```
def standardiser_automate(self):
    if self.est_standard():
        return

    transition_from_init = [trans for trans in self.transitions if trans[0] in self.init_etats]
    self.etats.append("Init")
    self.nb_etats += 1
    self.init_etats = ["Init"]
    self.nb_init_etats = 1

    for trans in transition_from_init:
        self.transitions.append(('Init', trans[1], trans[2]))
        self.nb_transitions += 1
```


g - Déterminisation de l'automate

```
def determiniser_automate(self):
    if self.est_deterministe():
        return

    transition_table = self.transitions
    new_transitions = []
    new_etats = []
    new_init_etats = []
    new_term_etats = []

    # Etats initiaux
    init_etat = ""

    for i in self.init_etats:
        init_etat += str(i)

    new_init_etats.append(init_etat)
    new_etats.append(init_etat)
    newetat_ind = 0
    while newetat_ind < len(new_etats):

        etat = new_etats[newetat_ind]

        new_trans = self.get_transitions_from_etat(etat)
        for trans in new_trans.items():
            new_transitions.append([etat, trans[0], trans[1]])
            # Ajout de l'état d'arrivée si nouveau
            if trans[1] not in new_etats and trans[1] != '':
                new_etats.append(trans[1])
                # Test si état final
                for i in [*trans[1]]:
                    if i in self.term_etats:
                        new_term_etats.append(trans[1])

        newetat_ind += 1

    self.transitions = new_transitions
    self.etats = new_etats
    self.init_etats = new_init_etats
    self.term_etats = new_term_etats
    self.nb_etats = len(new_etats)
    self.nb_init_etats = len(new_init_etats)
    self.nb_term_etats = len(new_term_etats)
    self.nb_transitions = len(new_transitions)
```

h - Compléter l'automate

```
def completer_automate(self):
    transition_table = self.transition_vers_tableau()
    if self.est_complet():
        return
    self.nb_etats += 1
    self.etats.append("P")

    for i in range(len(self.transitions)):
        for j in range(len(self.transitions[i])):
            if self.transitions[i][j] == '':
                self.transitions[i][j] = 'P'

    transition_table = self.transition_vers_tableau()

    for i in range(len(transition_table)):
        for j in range(len(transition_table[i])):
            if transition_table[i][j] == [' '] or transition_table[i][j] == ['']:
                transition_table[i][j] = ['P']
                # Obtenir le symbole correspondant à la valeur j
                symbole = self.get_key_from_value(self.symb, j)
                self.transitions.append((transition_table[i][0], symbole, "P"))
                self.nb_transitions += 1

    for i in range(self.nb_symb):
        symbole = self.get_key_from_value(self.symb, i + 1)
        self.transitions.append(("P", symbole, "P"))
        self.nb_transitions += 1
```

i - Automate Complémentaire

```
def language_comp(self):
    if self.est_deterministe() and self.est_complet():
        self.term_etats = list(
            set(self.etats) - set(self.term_etats))
        self.nb_term_etats = len(self.term_etats)

    elif self.est_deterministe() == False and self.est_complet() == False:
        print("L'automate doit être déterministe et complet !!! \n")

    elif self.est_deterministe() == True and self.est_complet() == False:
        print("L'automate doit être complet !!! \n")

    else:
        print("L'automate doit être déterministe !!! \n")
```

04 - Test du code

